

07

Services & Dependency Injection

© Gelişim · Junior → Mid-Level

🔗 Bu modülde ne öğreneceğiz?

- ✓ Service'in ne olduğunu ve neden component'lerden ayrı yaşadığını
- ✓ `@Service()` ile servis yazmayı ve `inject()` ile kullanmayı
- ✓ Dependency Injection'ın (DI) nasıl çalıştığını (singleton / kapsam)
- ✓ Durumu (state) bir servise taşıyıp component'ler arasında paylaşmayı
- ✓ Servise servis enjekte etmeyi ve `InjectionToken` ile değer sağlamayı

7.1 Service & DI Nedir?

Enjeksiyon & Token

7.2 @Service & inject

Mini Proje

7.3 Singleton & Kapsam

Özet

7.4 State Service

7.5

Ders 7.1

Service & Dependency Injection Nedir?

Şimdiye kadar mantığı ve veriyi hep component'lerin içine yazdın. Uygulama büyüdükçe bu sürdürülemez olur — aynı iş birden çok component'te tekrarlanır, veriyi paylaşmak zorlaşır. **Service** ve **Dependency Injection** tam bu derde deva.

Service nedir?

Service, bir işi ya da veriyi component'ten ayrı tutan, yeniden kullanılabilir bir sınıftır. Component'in görevi ekranı yönetmektir; “verileri getir”, “sepeti tut”, “günlük tut” gibi işler ise servislere taşınır. Böylece aynı mantık birçok component tarafından paylaşılır.

🔗 Metafor

Component bir **garson** (müşteriyle ilgilenir, masayı yönetir); service ise **mutfak** (asıl işi yapar). Garson her yemeği kendi pişirmez; mutfaktan ister. Aynı mutfak birçok garsona hizmet verir.

Dependency Injection (DI) nedir?

Bir component bir servise “ihtiyaç duyar” (ona **bağımlıdır**). **Dependency Injection, bu ihtiyacı senin elle oluşturman yerine Angular'ın sana hazır biçimde vermesidir.** Yani `new TodoService()` yazıp kendin kurmezsin; “bana TodoService lazım” dersin, Angular getirir.

Neden bu daha iyi? Çünkü Angular servisin **tek bir örneğini** (singleton) tutar ve ihtiyaç duyan herkese **aynı** örneği verir. Böylece bir component'in sepete eklediğini başka bir component de görür — çünkü ikisi de aynı servise bakar.

✓ Bu Dersten Çıkarılacaklar

- ✓ Service = işi/veriyi component'ten ayıran, paylaşılabilir sınıf.
- ✓ Component ekranı yönetir; service asıl işi/veriyi tutar (garson vs mutfak).
- ✓ DI = ihtiyaç duyduğun servisi elle kurmak yerine Angular'ın sana vermesi.
- ✓ Angular tek örneği (singleton) herkese verir → durum paylaşılır.

Ders 7.2

🔗 @Service() ve inject()

Bir servis yazmak ve kullanmak iki adımdır: sınıfı `@Service()` ile işaretlemek, sonra ihtiyaç duyan yerde `inject()` ile almak.

1) Servisi yaz: @Service()

Sınıfın başına `@Service()` koyarsın. Bu kadar — ekstra ayar gerekmez. Angular onu otomatik olarak **uygulama genelinde tek örnek** (kök singleton) yapar.

TYPESCRIPT

```
import { Service, signal } from '@angular/core';

@Service()
export class CounterService {
  private readonly _count = signal(0);
  readonly count = this._count.asReadonly();

  increment(): void {
    this._count.update((n) => n + 1);
  }
}
```

📌 `@Service()` ne yapar?

Sınıfı DI sistemine “enjekte edilebilir” olarak tanıtır ve varsayılan olarak **kök singleton** yapar (ayrı bir `providedIn` yazmana gerek yok). Ayrıca **constructor injection**'ı **desteklemez**; bağımlılıkları `inject()` ile alırsın — bu, modern ve tek yönlü bir yol sunar.

2) Servisi kullan: `inject()`

İhtiyaç duyan component (ya da başka bir servis) içinde `inject()` ile alırsın. Alan başlatıcısında (field initializer) çağırırsın:

TYPESCRIPT

```
import { Component, inject } from '@angular/core';
import { CounterService } from './counter-service';

@Component({
  selector: 'app-counter',
  template: `
    <p>Sayı: {{ counter.count() }}</p>
    <button (click)="counter.increment()">Artır</button>
  `,
})
export class Counter {
  protected readonly counter = inject(CounterService);
}
```

Dikkat: hiçbir yerde `new CounterService()` yazmadın. “Bana CounterService ver” dedin (`inject(CounterService)`), Angular hazır örneği getirdi. Servisin signal'lerini ve metotlarını doğrudan şablonda kullanabilirsin.

⚠ Neden `inject()` , constructor değil?

`@Service()` ile yazılan servislerde bağımlılıkları **constructor parametresiyle değil**, `inject()` fonksiyonuyla alırsın. Bu, alanları bir arada ve okunur tutar; ayrıca signal'lerle aynı zihinsel modele oturur.

🔗 Mini Quiz

1. `@Service()` bir sınıfa ne kazandırır?
↳ Onu DI'ya enjekte edilebilir yapar ve varsayılan kök singleton olarak sağlar.
2. Bir servisi component'te nasıl alırsın?
↳ `inject(ServisAdı)` ile (alan başlatıcısında).
3. Servisi `new` ile kurmalı mısın?
↳ Hayır; Angular tek örneği verir. Elle `new` kurmak paylaşımı bozar.

✗ Sık Yapılan Hatalar

- ✗ Servisi `new` ile kurmak. Paylaşılan tek örneği atlarsın; `inject()` kullan.
- ✗ Constructor injection beklemek. `@Service()` `inject()` ister.
- ✗ Durumu `public` signal olarak açmak. `private` tut, dışarı `asReadonly()` ver.

✓ Bu Dersten Çıkarılacaklar

- ✓ `@Service()` : sınıfı enjekte edilebilir + kök singleton yapar (ekstra ayar yok).
- ✓ `inject(ServisAdı)` : ihtiyaç duyan yerde servisi alır (constructor değil).
- ✓ Elle `new` yok; Angular tek örneği herkese verir.
- ✓ Servisin signal'leri ve metotları şablonda doğrudan kullanılabilir.

Ders 7.3

🎯 Singleton mı, Kapsamlı mı?

Varsayılan olarak servis **tek örnektir** (kök singleton) — tüm uygulama aynı örneği paylaşır. Çoğu servis için istediğin budur. Ama bazen bir servisin ömrünü belirli bir component'e bağlamak istersin.

Varsayılan: kök singleton

Sade `@Service()` → uygulama genelinde tek örnek. Sepet, kullanıcı bilgisi, tema gibi **paylaşılan** durumlar için doğru seçim: nerede `inject` edersen et, hep aynı veriye bakarsın.

Component kapsamı: `@Service({ autoProvided: false })`

Bir servisin her component örneğiyle **ayrı bir kopya** olarak yaşamasını istersen (örneğin yalnızca o component'e ait geçici UI durumu), `autoProvided: false` dersin ve onu o component'in `providers` dizisine eklersin:

```
import { Service } from '@angular/core';

@Service({ autoProvided: false })
export class WizardStateService {
  // ...yalnızca bir component ağacına ait durum
}
```

```
@Component({
  selector: 'app-wizard',
  providers: [WizardStateService], // bu component'e (ve çocuklarına) özel örnek
  // ...
})
export class Wizard {
  private readonly state = inject(WizardStateService);
}
```

Artık her `<app-wizard>` kendi `WizardStateService` örneğini alır; biri diğerinin durumunu görmez.

İhtiyaç	Seçim
Uygulama genelinde paylaşılan durum (sepet, kullanıcı, tema)	<code>@Service()</code> (varsayılan)
Yalnızca bir component'e ait, izole durum	<code>@Service({ autoProvided: false })</code> + <code>providers</code>

🔗 Önce singleton düşün

Component kapsamı gerçek bir mimari karardır; “her ihtimale karşı” diye kullanma. Servislerin çoğu paylaşılan olmalıdır (varsayılan). Kapsamı yalnızca **izole olması gereken** durum için bilinçli seç.

🔗 Mini Quiz

- Sade `@Service()` kaç örnek üretir?
↳ Bir tane — uygulama genelinde paylaşılan kök singleton.
- Bir servisi yalnızca bir component'e özel yapmak için ne yaparsın?

↳ `autoProvided: false` + component'in `providers` dizisine ekleme.

3. Sepet servisi hangi kapsamda olmalı?

↳ Kök singleton (varsayılan) — her yerden aynı sepete erişilsin.

✓ Bu Dersten Çıkarılacaklar

- ✓ Varsayılan `@Service()` = kök singleton (paylaşılan tek örnek).
- ✓ `@Service({ autoProvided: false })` + `providers` = component kapsamlı izole örnek.
- ✓ Paylaşılan durumlar singleton; izole UI durumu kapsamlı.
- ✓ Kapsamı bilinçli seç; varsayılan çoğu zaman doğrudur.

Ders 7.4

State Service: Durumu Paylaşmak

Modülün en pratik kısmı: bir **durumu (state)** servise taşıyıp birçok component'in paylaşmasını sağlamak. `ToDo` ve ürün uygulamalarında bunu component içinde yapmıştın; şimdi doğru yere — servise — taşıyoruz.

Desen: `private signal` + `asReadOnly()` + metotlar + `computed`

Durumu servis içinde `private` bir signal'de tutarsın; dışarıya yalnızca **okunur** halini (`asReadOnly()`) ve **metotları** açarsın. Böylece durum yalnızca servisin izin verdiği yollarla değişir — güvenli ve öngörülebilir.

TYPESCRIPT

```
import { Service, signal, computed } from '@angular/core';
import { Product } from '../models/product';

@Service()
export class CartService {
  // 1) Durum: yalnızca servis değiştirebilsin
  private readonly _items = signal<Product[]>([]);

  // 2) Dışarıya okunur hali
  readonly items = this._items.asReadOnly();

  // 3) Türetilmiş değerler
  readonly count = computed(() => this._items().length);
  readonly total = computed(() =>
    this._items().reduce((sum, p) => sum + p.price, 0),
  );

  // 4) Değiştirme yalnızca metotlarla (immutable!)
```

```

add(product: Product): void {
  this._items.update((list) => [...list, product]);
}

remove(id: number): void {
  this._items.update((list) => list.filter((p) => p.id !== id));
}

clear(): void {
  this._items.set([]);
}
}

```

Paylaşım: iki component, aynı sepet

Artık farklı component'ler aynı servisi **inject** eder ve **aynı** sepete bakar. Ürün listesi ekler, sepet özeti anında görür — çünkü ikisi de aynı singleton'a bağlıdır:

TYPESCRIPT

```

// Ürün listesi: sepete ekler
export class ProductList {
  private readonly cart = inject(CartService);
  addToCart(p: Product) { this.cart.add(p); }
}

// Sepet özeti: aynı sepeti okur
export class CartSummary {
  protected readonly cart = inject(CartService);
  // şablonda: cart.count(), cart.total(), cart.items()
}

```

🔗 Neden **asReadonly()** ?

Durumu doğrudan **public** yazılabilir signal olarak açsaydın, herhangi bir component onu izinsiz değiştirebilirdi ve kaynağı bulmak zorlaşırdı. **asReadonly()** + metotlar ile **tek değişim kapısı** servistir. Bu, önceki modüllerdeki immutability alışkanlığının servise taşınmış halidir.

🔗 Mini Quiz

1. Durumu serviste neden **private** signal'de tutarsın?
↳ Dışarıdan izinsiz değişmesin diye; dışarı **asReadonly()** + metotlar açılır.
2. Türetilmiş değerleri (toplam, adet) nasıl verirsin?

↳ `computed` ile (kaynak signal'den türetilir, otomatik güncel).

3. İki component aynı sepeti nasıl paylaşır?

↳ İkisi de aynı singleton servisi `inject` eder → aynı veriye bakarlar.

× Sık Yapılan Hatalar

- × Durumu `public` yazılabilir signal yapmak. Değişim kontrolünü kaybedersin; `private` + `asReadonly()`.
- × Diziyi mutasyonla değiştirmek (`push`). Immutable güncelle (`[...]` / `filter`).
- × Toplamı ayrı signal'de elle güncellemek. `computed` kullan.

✓ Bu Dersten Çıkarılacaklar

- ✓ Durumu servise taşı: `private signal` + `asReadonly()` + metotlar + `computed`.
- ✓ Tek değişim kapısı servistir → güvenli, öngörülebilir.
- ✓ Aynı singleton'ı `inject` eden component'ler durumu paylaşır.
- ✓ Immutability kuralı serviste de geçerli (yeni referans ver).

Ders 7.5

◀ ▶ Servise Servis Enjekte Etmek & InjectionToken

Servisler yalnızca component'lere değil, **birbirlerine de** enjekte edilir. Ayrıca sınıf olmayan değerleri (yapılandırma gibi) DI ile sağlamanın bir yolu vardır: `InjectionToken`.

Servise servis enjekte etmek

Bir servis, işini yaparken başka bir servise ihtiyaç duyabilir — aynı `inject()` ile alır:

TYPESCRIPT

```
@Service()
export class OrderService {
  private readonly cart = inject(CartService);
  private readonly logger = inject(LoggerService);

  checkout(): void {
    this.logger.log('Sipariş oluşturuluyor...');
    const items = this.cart.items();
    // ...
  }
}
```

InjectionToken : sınıf olmayan değerleri sağlamak

Bazen enjekte etmek istediğin şey bir sınıf değildir — bir API adresi, bir yapılandırma nesnesi gibi. Bunlar için bir **token** oluşturursun; DI ona bir değer bağlar:

TYPESCRIPT

```
import { InjectionToken } from '@angular/core';

export const API_BASE_URL = new InjectionToken<string>('API_BASE_URL');
```

TYPESCRIPT

```
// app.config.ts – token'a değer bağla
providers: [
  { provide: API_BASE_URL, useValue: 'https://dummyjson.com' },
],
```

TYPESCRIPT

```
// kullanırken
export class ProductService {
  private readonly baseUrl = inject(API_BASE_URL); // 'https://dummyjson.com'
}
```

📌 Ne zaman gerekir?

InjectionToken 'ı çoğunlukla **yapılandırma değerleri** için kullanırsın (API adresi, özellik bayrakları, ortam ayarları). Böylece bu değerleri tek yerden yönetir ve test ederken kolayca değiştirebilirsin. İleri bir konudur; şimdilik varlığını bilmen yeter.

💡 Mini Quiz

1. Bir servis başka bir servise nasıl ulaşır?
↳ `inject(DiğerServis)` ile — aynı component'teki gibi.
2. **InjectionToken** ne için vardır?
↳ Sınıf olmayan değerleri (API adresi, yapılandırma) DI ile sağlamak için.
3. Token'a değer nerede bağlanır?
↳ `providers` dizisinde `{ provide: TOKEN, useValue: ... }` ile.

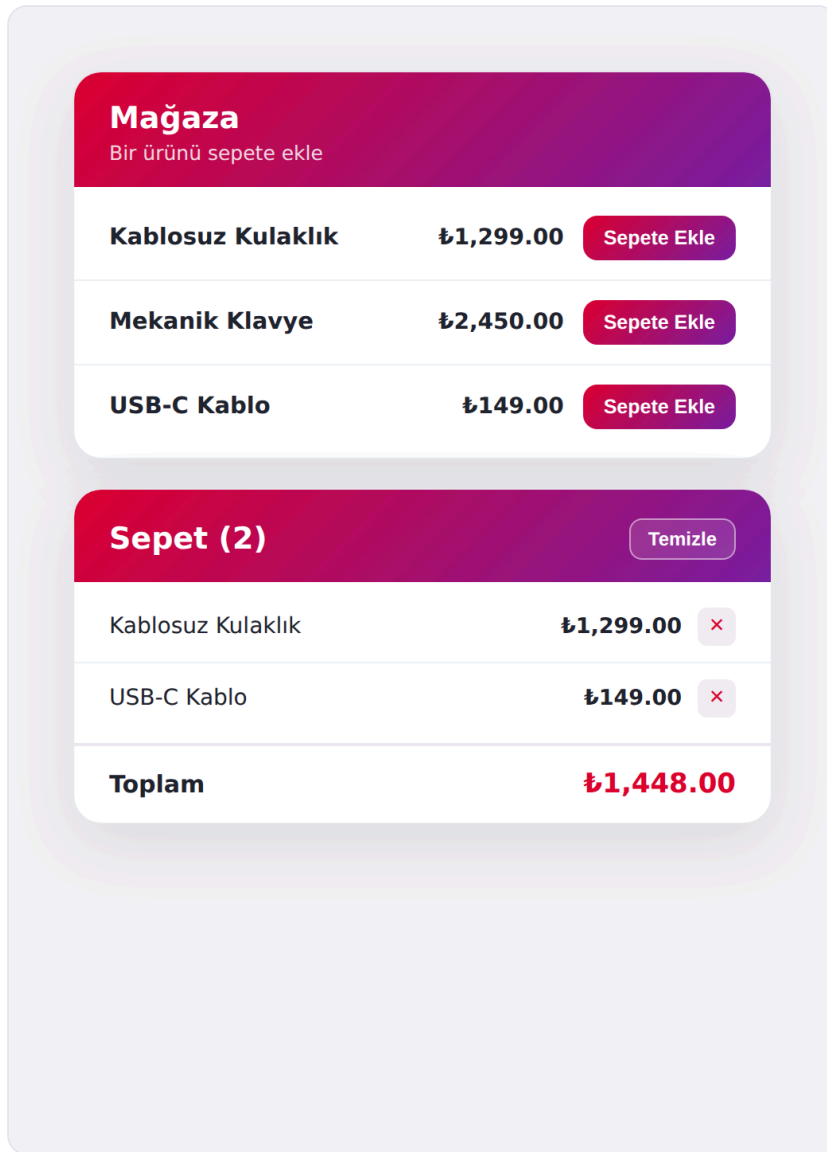
✓ Bu Dersten Çıkarılacaklar

- ✓ Servisler birbirine de `inject()` ile enjekte edilir.
- ✓ `InjectionToken` sınıf olmayan değerleri (yapılandırma) DI'ya taşır.
- ✓ Token'a değer `providers` 'da `{ provide, useValue }` ile bağlanır.
- ✓ Token çoğunlukla yapılandırma için; ileri ama faydalı bir araç.

Bölüm

🚩 Mini Proje: Sepet (Shared State Service)

Bu modülün özünü gerçek bir senaryoda birleştir: bir **sepet**. Ürün listesi ürünleri sepete ekler; sepet özeti aynı anda adedi ve toplamı gösterir. İkisi de aynı **CartService** singleton'ına bağlıdır — DI'n gücü tam burada görünür.



Ürün listesi (Sepete Ekle) + paylaşılan sepet özeti (adet & toplam)

Özellikler / Gereksinimler

- **CartService** (`@Service()`): sepet öğelerini `private signal` 'de tut.

- **Ekle / Çıkar / Temizle** metotları (immutable güncelleme).
- **Türetilmiş:** `count` (adet) ve `total` (tutar) birer `computed`.
- **Ürün listesi** component'i: her ürün için “Sepete Ekle”.
- **Sepet özeti** component'i: aynı servisi `inject` edip adet/toplamı ve öğeleri göster.
- Bir yerde ekleyince diğerinin **anında** güncellenmesi (paylaşım kanıtı).

Angular tarafı (sende) — ipuçları

- **Servis:** `private _items = signal<Product[]>([])` + dışarı `asReadonly()`; `add/remove/clear` metotları.
- **Türetilmiş:** `count = computed(() => items().length)`, `total = computed(...)`.
- **Paylaşım:** hem listede hem özet component'inde `cart = inject(CartService)` — aynı singleton.
- **Kapsam:** sepet paylaşılan olmalı → sade `@Service()` (kök singleton), `autoProvided: false` DEĞİL.
- **Şablon:** `@for` ile öğeler, `cart.count()` / `cart.total()` ile özet; boş sepet için `@empty`.

📌 Materyaller

Statik referansı ayrıca verdim (JS yok): **index.html** (ürün listesi + sepet özeti bir arada), **styles.scss**, **styles.css** ve ekran görüntüsü. Ürünleri DummyJSON'dan `httpResource` ile de çekebilirsin (Modül 5) ya da sabit bir liste kullanabilirsin — bu modülün odağı **servis + DI**. Bitince göster, birlikte gözden geçirelim.

✓ Bu Dersten Çıkarılacaklar

- ✓ Paylaşılan durum için tek singleton servis (`@Service()`).
- ✓ İki component aynı servisi `inject` edip aynı sepete bakar.
- ✓ `private signal` + `asReadonly()` + metotlar + `computed` deseni.
- ✓ Bir yerde değişiklik → her yerde anında yansır (DI + signal).

Bölüm

🚩 Modül 7 Özeti

Artık mantığı ve durumu component'lerden ayırıp paylaşabiliyorsun — büyük uygulamaların temeli budur.

- **Service:** işi/veriyi component'ten ayıran paylaşılabılır sınıf.
- `@Service()` + `inject()`: kök singleton + constructor'sız enjeksiyon.
- **Kapsam:** varsayılan singleton; izole durum için `autoProvided: false` + `providers`.
- **State service:** `private signal` + `asReadonly()` + metotlar + `computed`.
- **İleri:** servise servis enjekte etme, `InjectionToken` ile değer sağlama.

📌 Sırada ne var?

Modül 8 — HTTP & Backend İletişimi: `provideHttpClient` , `HttpClient` , interceptor'lar ve `httpResource` 'un derinlemesine işlenişi — verileri bir servis üzerinden getirmenin tam hali.

Takıldığın her yeri sor — mini projeyi (sepet) yapıp gösterdiğinde birlikte gözden geçiririz. Kolay gelsin!